

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Катышев М.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 26.09.26

Москва, 2026

Постановка задачи

Вариант 9.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы. В файле записаны команды вида: «число число<endline>». Дочерний процесс производит деление первого числа команда, на последующие числа в команде, а результат выводит в стандартный поток вывода. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса.

Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- read(fd, buf, count) — читает до count байт из дескриптора fd в буфер buf. Возвращает число прочитанных байт, 0 — конец файла, -1 — ошибка. Используется в родителе для чтения имени файла из stdin и в ребёнке для чтения команд из stdin (подменённого на файл).
- write(fd, buf, count) — пишет до count байт из буфера buf в дескриптор fd. Возвращает число записанных байт, -1 — ошибка. Используется для вывода результата в stdout, сообщений об ошибках в stderr и записи данных в pipe.
- open(path, flags) — открывает файл по пути path с флагами flags (в работе — O_RDONLY, только чтение). Возвращает файловый дескриптор, -1 — ошибка. Используется родителем для открытия входного файла.
- close(fd) — закрывает файловый дескриптор fd, освобождая ресурс. Возвращает 0 при успехе, -1 при ошибке. Используется для закрытия ненужных дескрипторов после dup2 и по завершении работы с pipe и файлом.
- pipe(fd) — создаёт однонаправленный канал (pipe) и заполняет массив fd двумя дескрипторами: fd[0] — для чтения, fd[1] — для записи. Возвращает 0 при успехе, -1 при ошибке. Используется для передачи результата от дочернего процесса родительскому.
- fork() — создаёт дочерний процесс как копию текущего. Возвращает: родителю — PID дочернего процесса (> 0), дочернему — 0, при ошибке — -1. Используется для порождения дочернего процесса.
- dup2(oldfd, newfd) — делает дескриптор newfd копией oldfd, закрывая старый newfd. Возвращает новый дескриптор, -1 — ошибка. Используется в дочернем процессе: dup2(file, 0) подменяет stdin на файл, dup2(fd[1], 1) подменяет stdout на pipe.
- execl(path, arg0, ..., NULL) — заменяет текущий процесс программой по пути path. При успехе не возвращается; при ошибке возвращает -1. Используется для запуска программы child в дочернем процессе.

- `waitpid(pid, &status, options)` — ожидает завершения дочернего процесса с указанным `pid` и записывает статус в `status`. Возвращает PID завершившегося процесса, `-1` — ошибка. Используется родителем для ожидания ребёнка и получения кода его возврата.

В рамках лабораторной работы были написаны две программы на C++: `parent` и `child`.

Сначала в родительской программе реализовали чтение имени файла из стандартного потока ввода системным вызовом `read` и открытие этого файла на чтение через `open`. Затем создали канал (`pipe`) для передачи данных между процессами и с помощью `fork` породили параллельный дочерний процесс. В дочернем процессе аккуратно подменили стандартные потоки через `dup2`: `stdin` — на открытый файл, `stdout` — на конец канала для записи. После этого закрыли все ненужные дескрипторы, чтобы не оставалось лишних открытых ресурсов и канал корректно закрывался. Через `execl` дочерний процесс запустил отдельную программу `child`, которая читает команды из `stdin`, выполняет деление и пишет результат в `stdout`. Родитель при этом закрыл свой конец канала для записи, прочитал данные из канала для чтения и вывел их в терминал, а затем дождался завершения ребёнка через `waitpid`. Дополнительно была реализована проверка на деление на ноль: при обнаружении нулевого делителя дочерний процесс выводит сообщение об ошибке и завершается с кодом 1, а родитель, получив этот код через `waitpid`, также завершается с кодом 1. По ходу работы добавили обработку ошибок всех системных вызовов (`read`, `open`, `pipe`, `fork`, `execl`), вывели сообщения в `stderr` и продумали коды возврата. После написания код был собран компилятором `g++` с флагами `-Wall -Wextra` и протестирован на разных входных данных: обычное деление, одно число, деление на ноль, отсутствие чисел, несуществующий файл. В процессе отладки были исправлены ошибки: путаница в именах буфера, некорректное сравнение вместо присваивания, неточность вывода `float` из-за особенностей представления чисел с плавающей точкой. В итоге программа корректно обрабатывает все тестовые случаи и соответствует заданию.

Код программы

`parent.cpp`

```
#include <unistd.h>
#include <fcntl.h>
#include <cstring>
#include <sys/wait.h>

int main() {
    char file_name[256];

    ssize_t n = read(0, file_name, sizeof(file_name) - 1);

    if (n == -1) {
        const char* msg = "read error\n";
        write(2, msg, strlen(msg));
        return 1;
    }

    if (n == 0) {
        const char* msg = "no file name found\n";
        write(2, msg, strlen(msg));
        return 1;
    }
```

```

}

file_name[n] = '\0';
if (n > 0 && file_name[n - 1] == '\n') { file_name[n - 1] = '\0'; }

int file = open(file_name, O_RDONLY);
if (file == -1) {
    const char* msg = "open file error\n";
    write(2, msg, strlen(msg));
    return 1;
}

//const char* ok = "file open\n";
//write(1, ok, strlen(ok));

int fd[2];
if (pipe(fd) == -1){
    const char* msg = "pipe error\n";
    write(2, msg, strlen(msg));
    return 1;
}

pid_t pid = fork();

if (pid == -1) {
    const char* msg = "fork error\n";
    write(2, msg, strlen(msg));
    return 1;
}

if (pid == 0){
    //const char* msg = "child\n";
    //write(1, msg, strlen(msg));

    dup2(file, 0);
    dup2(fd[1], 1);
    close(fd[0]);
    close(fd[1]);
    close(file);

    execl("./child", "child", NULL);

    const char* msg = "exec error\n";
    write(2, msg, strlen(msg));
    return 1;
} else {
    //const char* msg = "parent\n";
    //write(1, msg, strlen(msg));

    close(fd[1]);
    close(file);

char result[4096];
ssize_t r;
while ((r = read(fd[0], result, sizeof(result))) > 0){
    write(1, result, r);
}
close(fd[0]);

```

```

int status;
waitpid(pid, &status, 0);

if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
    const char* msg = "child failed\n";
    write(2, msg, strlen(msg));
    return 1;
}
}

return 0;

```

child.cpp

```

#include <unistd.h>
#include <cstring>
#include <cstdlib>

int main() {
    char numbers[4096];
    ssize_t n = read(0, numbers, sizeof(numbers) - 1);
    if (n <= 0) return 0;

    numbers[n] = '\0';

    char* line = numbers;
    while (line != nullptr && *line != '\0') {
        char* nl = strchr(line, '\n');
        if (nl != nullptr) {
            *nl = '\0';
        }

        char* p = line;
        char* end;

        double result = strtod(p, &end);
        if (end == p) {
            if (nl == nullptr) break;
            line = nl + 1;
            continue;
        }
        p = end;

        while (true) {
            double x = strtod(p, &end);
            if (end == p) break;

            if (x == 0.0) {
                const char* msg = "division by zero\n";
                write(2, msg, strlen(msg));
                exit(1);
            }

            result /= x;
            p = end;
        }
    }
}

```

```

char out[64];
int len = 0;

if (result < 0) {
    out[len++] = '-';
    result = -result;
}

long int_part = (long)result;
char digits[32];
int t = 0;
if (int_part == 0) {
    digits[t++] = '0';
} else {
    while (int_part > 0) {
        digits[t++] = '0' + (int_part % 10);
        int_part /= 10;
    }
}
while (t > 0) {
    out[len++] = digits[--t];
}

out[len++] = '.';
double frac = result - (long)result;
for (int i = 0; i < 4; i++) {
    frac *= 10;
    int digit = (int)(frac + 1e-9);
    out[len++] = '0' + digit;
    frac -= digit;
}
out[len++] = '\n';

write(1, out, len);

if (nl == nullptr) break;
line = nl + 1;
}

return 0;
}

```

Протокол работы программы

Тестирование:

```

input.txt
1.0500
9.0000
-5.0000
0.3333
division by zero
child failed

```

Вывод

В ходе лабораторной работы были изучены и практически применены системные вызовы для работы с процессами и межпроцессного взаимодействия: `fork`, `execl`, `pipe`, `dup2`,

waitpid, read, write, open и close. Была разработана программа из двух взаимодействующих процессов, в которой родитель открывает файл, создаёт канал и порождает дочерний процесс, а дочерний процесс через перенаправление дескрипторов читает данные из файла, выполняет вычисления и передаёт результат родителю через канал. Освоены принципы перенаправления стандартных потоков ввода-вывода и обработки ошибок системных вызовов. Полученный результат подтверждает, что механизм fork + exec + dup2 + pipe является стандартным способом организации взаимодействия процессов в Unix-подобных системах.